TERRAFORM CHALLENGE - 2

Goal of the Exercise

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

- The service will be a simple web server that returns a static page.
- You can use **apache2** or **nginx** to serve a simple HTML file.
- The service should be **containerized** and stored in a **Docker registry**.
- The service should be deployed on a **single VM instance** as a docker container.
- The image should be pulled from the registry specified earlier.
- The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

You are to provide the following as part of your solution:

1. **Dockerfile** to build the image.
2. **Terraform codebase** to deploy the following:
    - Docker registry
    - VPC and all required networking resources
    - EC2 instance
    - Load Balancer

Notes

- Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.
- To push the image to the Docker registry, **build the image locally** before running the Terraform code.
- Once the image registry is created, use Terraform to **push the local image** to the registry.
- Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.
- For SSH access:
    - Use a locally created key.
    - Add it to the cloud provider key storage.
    - Attach it to the instance.
- Use **Terraform best practices** for file structure and code organization.

    - Use variables, locals, modules, etc. when appropriate.
- Include all `tfvars`, scripts, and other helper files used to deploy the application.
- You may use any cloud provider of your choosing, although **AWS is preferred**.
- Use a **free tier account** or a **trial account** to deploy the resources.

AWS + Terraform + Docker Microservice Deployment – Complete Step-by-Step Documentation

Practical Project
AWS + Terraform + Docker + EC2 + ECR + ALB

I will explain this project from the beginning, assuming you are starting with an empty folder on your Windows system.

The documentation will cover:

1. What exactly we are building.
2. Why we are using each AWS service.
3. How the complete architecture works.
4. Where to go in AWS Console and what to create.
5. Which folders and files to create in VS Code.
6. Complete Terraform code for each file.
7. Docker image creation and ECR image push.
8. EC2 creation and automatic Docker container deployment.
9. Load Balancer configuration.
10. Testing, troubleshooting and cleanup.

I will use AWS as the cloud provider, Windows PowerShell as the terminal, and Terraform for infrastructure provisioning.

PART 1 – Understand the project first

1.1 What is the actual task?

We need to deploy a simple web application using Docker containers on AWS.

Our application will display a static HTML page.

For example:

VK Microservice Application

Example page



This is not a large application. Our main objective is to demonstrate that we can build, store, deploy and expose a containerized application using Terraform.

1.2 What are we going to create?

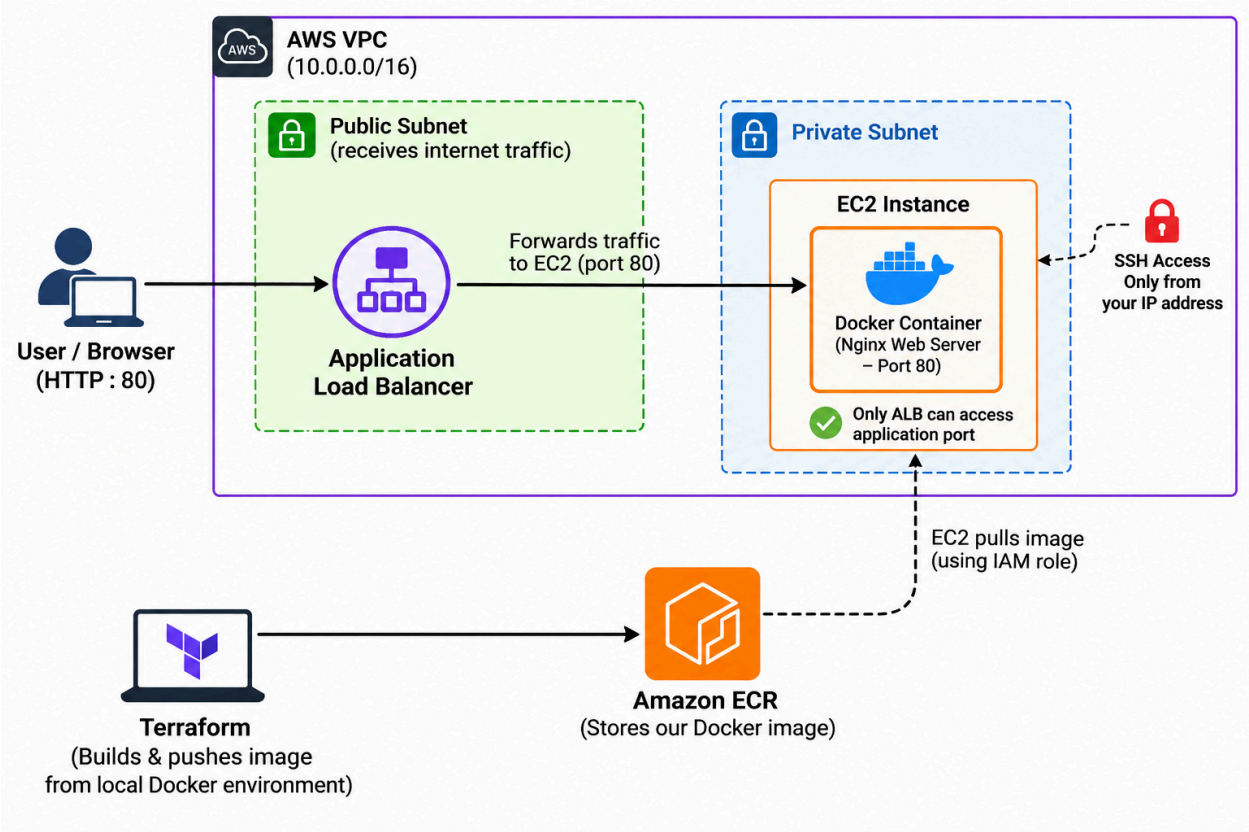
Component	What we create	Why we need it
Dockerfile	Application image instructions	To package the web server
Docker	Container image	To run our application
ECR	AWS container registry	To store Docker images

VPC	Private AWS network boundary	To isolate infrastructure
Internet Gateway	Internet connectivity	For public resources
Subnets	Network divisions	To place AWS resources
Route Table	Network routing	To direct traffic
Security Groups	Network firewall	To control access
EC2	Virtual machine	To run the Docker container
IAM Role	EC2 permissions	To allow EC2 to pull from ECR
Key Pair	SSH authentication	To connect to EC2
ALB	Application Load Balancer	To receive user requests and forward them to EC2
Terraform	Infrastructure as Code	To create all AWS resources

1.3 Understand the architecture

This is the architecture we are going to implement.

AWS Microservice Architecture



How does the complete process work?

1. We create a static HTML application.
2. We create a Dockerfile.
3. We build a Docker image on our local Windows machine.
4. Terraform creates an AWS ECR repository.
5. Terraform pushes our local Docker image into ECR.
6. Terraform creates the VPC and networking resources.
7. Terraform creates an EC2 instance.
8. EC2 installs Docker automatically.
9. EC2 authenticates with ECR and pulls the image.
10. EC2 starts the Docker container.
11. Terraform creates an Application Load Balancer.
12. Users access the application using the Load Balancer DNS name.

Important security decision: We will place the EC2 instance in a public subnet with a public IP to simplify the lab and avoid the cost and complexity of a NAT Gateway. Its security group will block direct application access from the internet. Port 80 will accept traffic only from the ALB security group. SSH will accept traffic only from the IP/CIDR you specify. Thus, the web service itself is accessible only through the Load Balancer.

For a production environment, a private EC2 subnet with appropriate VPC endpoints or NAT connectivity would be preferable.

PART 2 – Prerequisites

Before creating anything, check the following.

2.1 Install the required software

You need these tools on your Windows machine.

Software	Purpose
VS Code	Write Terraform and Docker files
Terraform	Provision AWS infrastructure
AWS CLI	Connect your computer to AWS
Docker Desktop	Build Docker images locally
Git	Optional for version control

2.2 Check installations

Open PowerShell.

Run these commands separately:

Check Terraform

```
None
terraform --version
```

```
PS C:\Users\chvka\vk-ai> terraform --version
Terraform v1.16.6
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.17.1. You can update by downloading from https://developer.hashicorp.com/terraform/install
PS C:\Users\chvka\vk-ai>
```

Check AWS CLI

```
None
aws --version
```

```
PS C:\Users\chvka\vk-ai> aws --version
aws-cli/2.36.30 Python/3.14.6 Windows/11 exe/AMD64
PS C:\Users\chvka\vk-ai>
```

Check Docker

```
None
docker --version
```

```
PS C:\Users\chvka\vk-ai> docker --version
Docker version 29.8.2, build 7fc2dff
PS C:\Users\chvka\vk-ai>
```

Check Git

```
None
git --version
```

```
PS C:\Users\chvka\vk-ai> git --version
git version 2.56.0.windows.4
PS C:\Users\chvka\vk-ai>
```

If all commands return version information, the tools are installed.

2.3 Start Docker Desktop

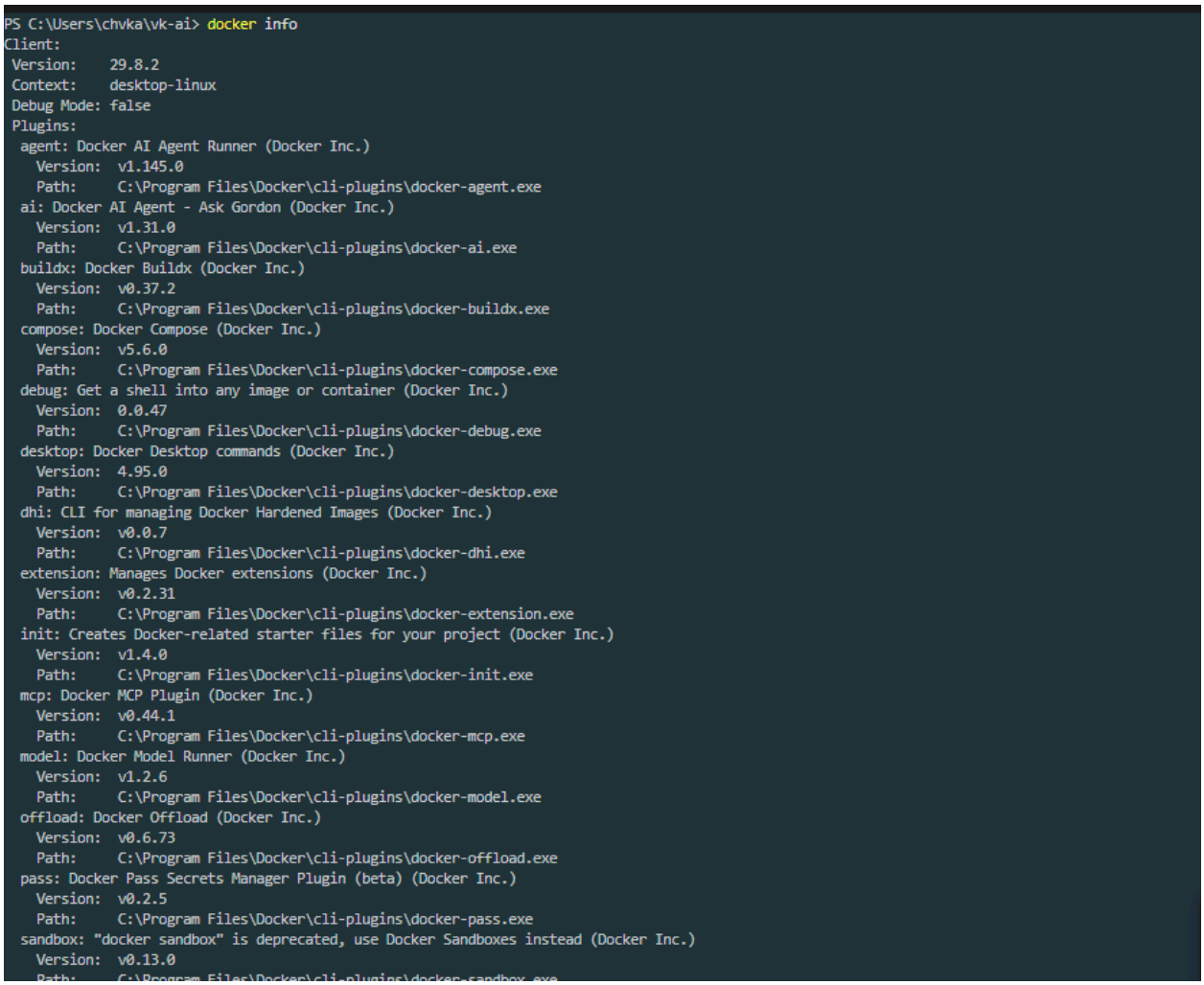
1. Open Windows Start Menu.
2. Search for Docker Desktop.
3. Open Docker Desktop.
4. Wait until Docker Engine is running.
5. Open PowerShell.

Run:

```
None
docker info
```

If Docker is running, you should see information about the Docker server.

If it shows an error such as `Cannot connect to the Docker daemon`, start Docker Desktop before proceeding.



PART 3 – AWS Account Configuration

3.1 Configure AWS CLI

Open PowerShell.

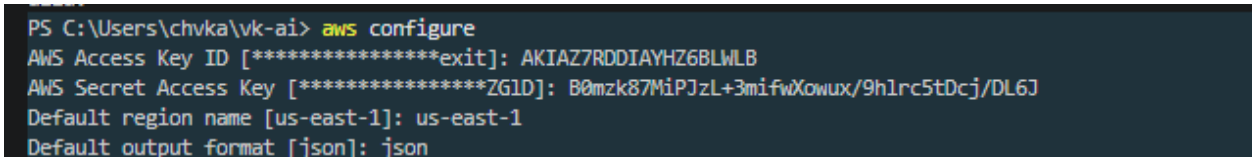
Run:

```
None
aws configure
```

Enter your AWS credentials:

```
None
AWS Access Key ID: YOUR_ACCESS_KEY
AWS Secret Access Key: YOUR_SECRET_KEY
Default region name: us-east-1
Default output format: json
```

We will use `us-east-1` throughout this project.



Why are we configuring AWS CLI?

Terraform needs AWS credentials to communicate with AWS and create resources.

The AWS CLI configuration makes your credentials available to Terraform through the AWS SDK credential chain.

Do not put AWS secret keys directly into Terraform files or Git repositories.

3.2 Verify AWS connection

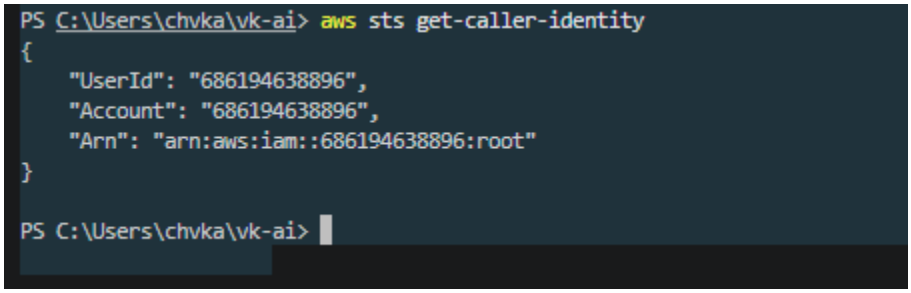
Run:

```
None
aws sts get-caller-identity
```

Expected result:

```
None
{
    "UserId": "YOUR_USER_ID",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/your-user"
}
```

If this command works, your local computer can communicate with AWS.



3.3 Create an SSH key locally

We need an SSH key to connect to EC2.

Open PowerShell:

```
None
mkdir C:\Users\$env:USERNAME\.ssh -Force
```

Create the key:

```
None
ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\microservice-key"
```

When prompted for a passphrase, you can press Enter twice for this lab.

```
PS C:\Users\chvka\vk-ai> mkdir C:\Users\$env:USERNAME\.ssh -Force


    Directory: C:\Users\chvka


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        30-10-2026     12:36                .ssh


PS C:\Users\chvka\vk-ai> ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\microservice-key"
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\chvka\.ssh\microservice-key
Your public key has been saved in C:\Users\chvka\.ssh\microservice-key.pub
The key fingerprint is:
SHA256:2eGYZg52j7S1Njpzm7YqTsJ1DY6H02DNiqWUQp91KAg chvka@karthik
The key's randomart image is:
+--[ED25519 256]--+
|E..  ...         |
|...oo+.          |
|. oo= +   .      |
| o = B o * .     |
|  o = O S +      |
|   . * B = .     |
|    o . + =      |
|     o. oooo     |
|     ...o*+o     |
+----[SHA256]-----+
PS C:\Users\chvka\vk-ai>
```

This creates two files:

File	Purpose
microservice-key	Private key
microservice-key.pub	Public key

Important:

- Private key stays on your local computer.
- Public key is uploaded to AWS using Terraform.
- Never upload or share the private key.

We will use the public key in our Terraform configuration.

PART 4 – Create the Project Folder Structure

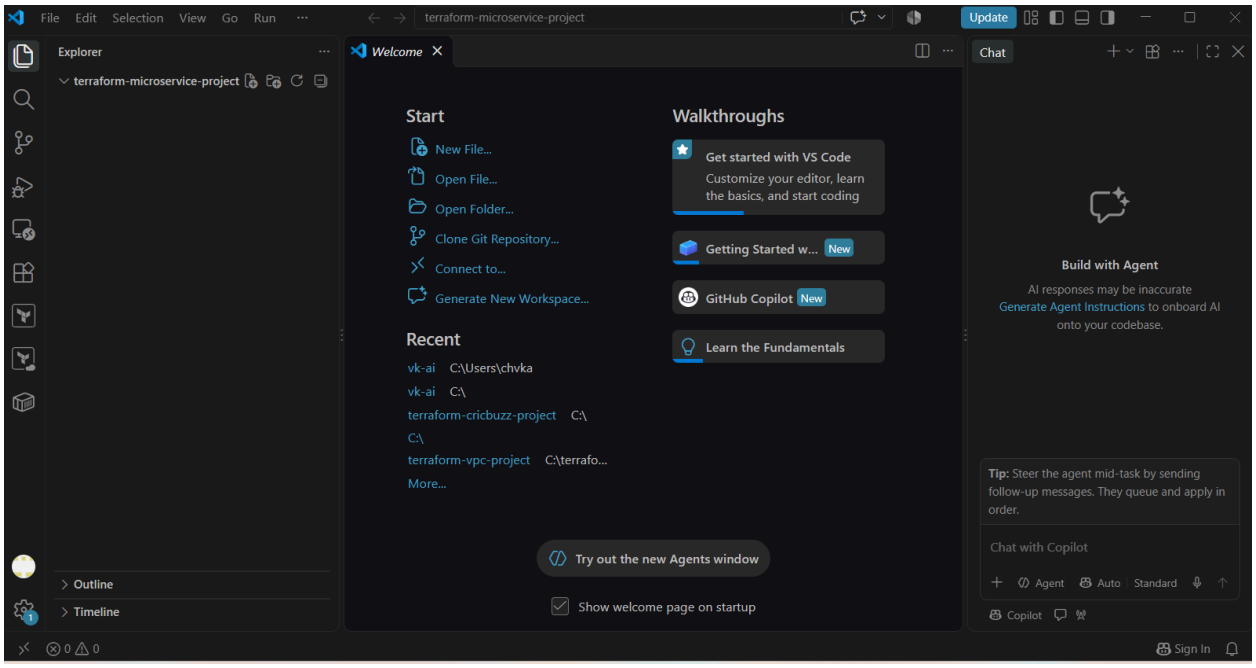
Now we will create the project in VS Code.

4.1 Create the main folder

Open PowerShell:

```
None
cd C:\
mkdir terraform-microservice-project
cd terraform-microservice-project
code .
```

This opens the project in VS Code.



4.2 Create the directories

Create the following folders:

```
None
terraform-microservice-project/
│
├── app/
│
├── terraform/
│
└── scripts/
```

Why do we need separate folders?

- `app/` – Contains the application HTML file and Dockerfile.
- `terraform/` – Contains all Terraform configuration files.
- `scripts/` – Contains the EC2 startup script.

4.3 Create all required files

Your final structure will look like this:

```
None
terraform-microservice-project/
│
├── app/
│   ├── index.html
│   └── Dockerfile
│
├── scripts/
│   └── user_data.sh.tftpl
│
└── terraform/
    ├── versions.tf
    ├── providers.tf
    ├── variables.tf
    ├── locals.tf
    ├── networking.tf
    ├── security.tf
    ├── ecr.tf
```

```
    ├── iam.tf
    ├── ec2.tf
    ├── alb.tf
    ├── outputs.tf
    └── terraform.tfvars
```

We are separating files according to their responsibilities.

For example, all networking resources will be in `networking.tf`, and all EC2 resources will be in `ec2.tf`.

PART 5 – Create the Web Application

5.1 Create index.html

Where to go: VS Code → app folder → New File → `index.html`

Paste this code:

None

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>VK Microservice</title>

    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
```

```
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #101a38, #243b67);
    color: white;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 25px;
}

.container {
    max-width: 850px;
    width: 100%;
    background: rgba(255, 255, 255, 0.08);
    border: 1px solid rgba(255, 255, 255, 0.2);
    border-radius: 22px;
    padding: 55px 35px;
    text-align: center;
    box-shadow: 0 15px 45px rgba(0, 0, 0, 0.3);
}

.logo {
    font-size: 48px;
    font-weight: bold;
    color: #65c7ff;
    margin-bottom: 15px;
}

h1 {
    font-size: 36px;
    margin-bottom: 18px;
}

.description {
```

```
        color: #d3e2ff;
        font-size: 17px;
        line-height: 1.6;
        margin-bottom: 35px;
    }

    .section-title {
        font-size: 21px;
        margin-bottom: 22px;
    }

    .technologies {
        display: flex;
        flex-wrap: wrap;
        justify-content: center;
        gap: 15px;
        margin-bottom: 35px;
    }

    .tech {
        background: rgba(255, 255, 255, 0.12);
        border: 1px solid rgba(255, 255, 255, 0.2);
        padding: 15px 22px;
        border-radius: 12px;
        font-size: 15px;
        font-weight: bold;
    }

    .status {
        display: inline-block;
        background: #123e32;
        color: #65f5b0;
        border: 1px solid #28b879;
        padding: 15px 25px;
        border-radius: 30px;
        font-weight: bold;
```

```
            font-size: 16px;
        }

        .footer {
            margin-top: 35px;
            color: #aebfdf;
            font-size: 13px;
        }

        @media (max-width: 600px) {
            h1 {
                font-size: 27px;
            }

            .container {
                padding: 40px 20px;
            }

            .tech {
                width: 100%;
            }
        }
    </style>
</head>

<body>

    <div class="container">

        <div class="logo">VK</div>

        <h1>Welcome to VK Microservice!</h1>

        <p class="description">
            This application is successfully deployed using
            modern DevOps and AWS cloud technologies.
```

```
        </p>

        <h2 class="section-title">Technologies Used</h2>

        <div class="technologies">
            <div class="tech">Docker</div>
            <div class="tech">Terraform</div>
            <div class="tech">AWS EC2</div>
            <div class="tech">AWS ECR</div>
            <div class="tech">Application Load Balancer</div>
        </div>

        <div class="status">
            ✓ Application Running Successfully
        </div>

        <div class="footer">
            VK Microservice | Powered by AWS & DevOps
        </div>

    </div>

</body>
</html>
```

Why are we creating this file?

This is our actual web page.

When a user opens the application, Nginx will serve this HTML file.

We are not developing a complex application because the assignment focuses on cloud infrastructure and deployment.

5.2 Create the Dockerfile

Where to go: VS Code → app folder → New File → `Dockerfile`

Paste:

```
None
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Explanation of each instruction

Instruction	Explanation	Why are we using it?
FROM nginx:alpine	Downloads the lightweight Nginx base image	To use Nginx as our web server
COPY	Copies our HTML file into the container	To display our application page
EXPOSE 80	Documents the application's listening port	Our web server uses port 80
CMD	Starts Nginx in the foreground	Keeps the container running

How does this work?

Our Dockerfile tells Docker:

1. Download the Nginx image.
2. Copy our HTML file into the Nginx web directory.
3. Configure the application to use port 80.
4. Start the Nginx web server.

PART 6 – Build and Test the Docker Image Locally

Before connecting to AWS, we should test the application locally.

6.1 Open PowerShell

Run:

```
None
cd C:\terraform-microservice-project\app
```

6.2 Build the Docker image

Run:

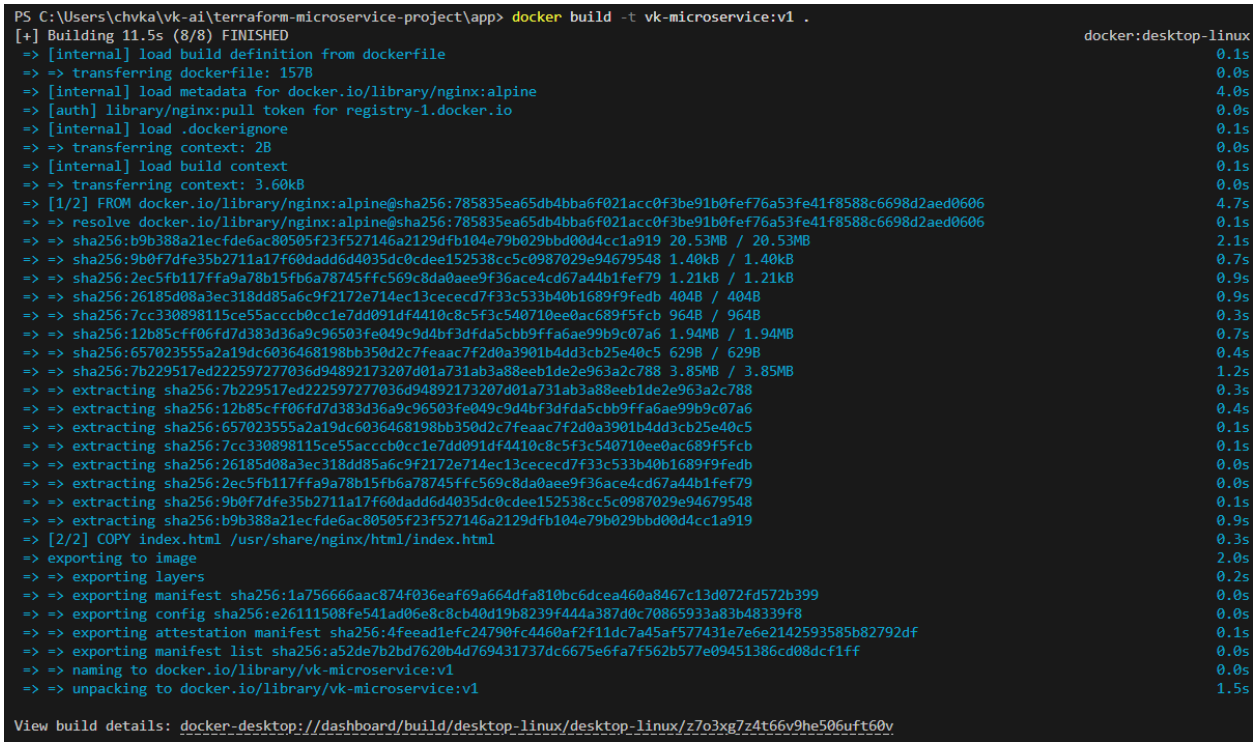
```
None
docker build -t vk-microservice:v1 .
```

Explanation

- `docker build` – Builds an image.
- `-t` – Assigns a name and tag.
- `vk-microservice` – Image name.
- `v1` – Image version.
- `.` – Uses the current directory as the build context.

Expected output:

```
None
Successfully built IMAGE_ID
Successfully tagged vk-microservice:v1
```



6.3 Check the Docker image

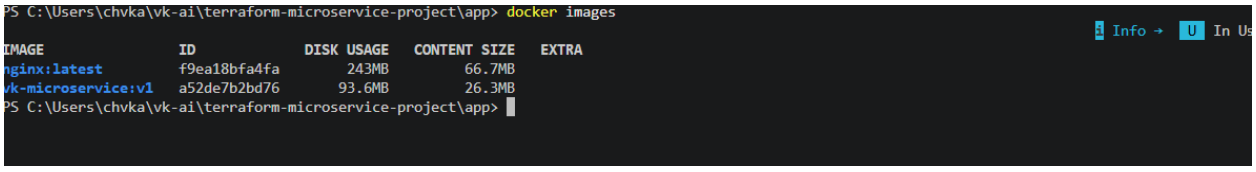
None

```
docker images
```

You should see:

None

```
REPOSITORY          TAG
vk-microservice     v1
```



6.4 Run the container locally

```
None
docker run -d --name vk-test -p 8080:80 vk-microservice:v1
```

What does this command do?

Part	Meaning
docker run	Starts a container
-d	Runs in background
--name vk-test	Gives the container a name
-p 8080:80	Maps local port 8080 to container port 80
vk-microservice:v1	Uses our image

6.5 Test the application

Open your browser:

```
None
http://localhost:8080
```

You should see:

Welcome to VK Microservice!



Local Docker test completed

6.6 Stop the test container

After testing:

```
None
docker stop vk-test
```

Remove it:

```
None
docker rm vk-test
```

Important: Do not delete the Docker image. We need it for the ECR push.

PART 7 – Create the Terraform Configuration

Now we will start creating AWS infrastructure.

We will create these resources:

1. VPC.
2. Internet Gateway.
3. Two public subnets.
4. Route table.
5. Security groups.
6. ECR repository.
7. IAM role and instance profile.
8. EC2 instance.
9. Application Load Balancer.
10. Target group and listener.

7.1 Create versions.tf

Location: `terraform/versions.tf`

```
None
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }

    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}
```

Explanation

This file defines which Terraform providers our project needs.

AWS provider: Communicates with AWS and creates AWS resources.

Docker provider: Communicates with your local Docker engine and manages Docker images.

Why do we need both?

Because we have two different responsibilities:

- Docker provider builds and pushes the image.
- AWS provider creates the AWS infrastructure.

7.2 Create providers.tf

Location: `terraform/providers.tf`

```
None
provider "aws" {
  region = var.aws_region
}

data "aws_ecr_authorization_token" "token" {}

provider "docker" {
  registry_auth {
    address  =
data.aws_ecr_authorization_token.token.proxy_endpoint
    username = data.aws_ecr_authorization_token.token.user_name
    password = data.aws_ecr_authorization_token.token.password
  }
}
```

Explanation

The AWS provider connects Terraform to AWS.

The Docker provider connects Terraform to the local Docker engine.

The ECR authorization token provides the temporary credentials required to push the Docker image to AWS ECR.

Important: The AWS identity used by your local Terraform process needs permissions for ECR repository creation, image upload and the other resources in this project.

7.3 Create variables.tf

Location: `terraform/variables.tf`

None

```
variable "aws_region" {
  description = "AWS region"
  type        = string
}

variable "project_name" {
  description = "Project name"
  type        = string
}

variable "vpc_cidr" {
  description = "VPC CIDR range"
  type        = string
}

variable "public_subnet_cidrs" {
  description = "Public subnet CIDR ranges"
  type        = list(string)
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "key_name" {
  description = "AWS EC2 key pair name"
  type        = string
}

variable "public_key_path" {
  description = "Local SSH public key path"
  type        = string
}

variable "ssh_allowed_cidrs" {
```

```
  description = "CIDR ranges allowed to SSH into EC2"
  type        = list(string)
}

variable "image_tag" {
  description = "Docker image version"
  type        = string
}
```

Why are we using variables?

Instead of hardcoding values in every resource, we define variables once and provide their values through `terraform.tfvars`.

For example:

- AWS region can be changed.
- EC2 instance type can be changed.
- VPC CIDR can be changed.
- SSH allowed IP can be changed.
- Docker image tag can be changed.

This makes our Terraform code reusable.

7.4 Create locals.tf

Location: `terraform/locals.tf`

```
None
locals {
  common_tags = {
    Project     = var.project_name
    Environment = "Dev"
    ManagedBy   = "Terraform"
  }

  image_name =
"${aws_ecr_repository.app.repository_url}:${var.image_tag}"
```

```
}
```

Explanation

Locals are used for reusable values inside Terraform.

Here we are defining common tags for AWS resources.

We are also creating the complete Docker image name using the ECR repository URL and image tag.

Example:

```
None
123456789012.dkr.ecr.us-east-1.amazonaws.com/vk-microservice:v1
```

PART 8 – Create VPC and Networking Resources

This is one of the most important parts of the assignment.

8.1 What is a VPC?

VPC stands for Virtual Private Cloud.

It is an isolated virtual network inside AWS.

We need a VPC because we should not deploy all resources without network isolation and access controls.

Our VPC CIDR:

```
None
10.0.0.0/16
```

We will create two public subnets.

Resource	CIDR
VPC	10.0.0.0/16
Public Subnet 1	10.0.1.0/24
Public Subnet 2	10.0.2.0/24

The two subnets will be in different Availability Zones.

Why?

An Application Load Balancer requires subnets in at least two Availability Zones.

8.2 Create networking.tf

Location: `terraform/networking.tf`

Paste:

```
None
data "aws_availability_zones" "available" {
  state = "available"
}

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-vpc"
```

```
  })
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-igw"
  })
}

resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)

  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidrs[count.index]
  availability_zone       =
data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-public-${count.index + 1}"
  })
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-public-rt"
  })
```

```
}

resource "aws_route_table_association" "public" {
  count = length(aws_subnet.public)

  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}
```

Understand each resource

1. aws_vpc

Creates our virtual network.

2. aws_internet_gateway

Connects the VPC to the internet.

3. aws_subnet

Creates two subnets in different Availability Zones.

4. aws_route_table

Defines where network traffic should go.

5. aws_route_table_association

Associates each public subnet with the route table.

Why do we need the Internet Gateway?

Our ALB must receive traffic from users on the internet.

Our EC2 also needs outbound connectivity to pull the ECR image.

We use the Internet Gateway for this lab.

PART 9 – Create Security Groups

9.1 What is a Security Group?

A Security Group is a virtual firewall attached to AWS resources.

It controls incoming and outgoing traffic.

We need two security groups.

Security Group	Purpose
ALB Security Group	Controls traffic entering the Load Balancer
EC2 Security Group	Allows traffic from ALB and restricted SSH users

Our traffic rules:

Resources	Port	Allowed source
ALB	80	0.0.0.0/0
EC2	80	ALB Security Group
EC2	22	Your specified IP address only

Notice that EC2 port 80 does not accept internet traffic directly.

9.2 Create security.tf

Location: `terraform/security.tf`

None

```
resource "aws_security_group" "alb_sg" {
  name        = "${var.project_name}-alb-sg"
  description = "Allow HTTP traffic to ALB"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "Allow HTTP from internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = local.common_tags
}

resource "aws_security_group" "ec2_sg" {
  name        = "${var.project_name}-ec2-sg"
  description = "Allow HTTP from ALB and SSH from approved IPs"
  vpc_id      = aws_vpc.main.id

  ingress {
    description     = "Allow HTTP only from ALB"
    from_port       = 80
    to_port         = 80
    protocol        = "tcp"
    security_groups = [aws_security_group.alb_sg.id]
  }
```

```
  dynamic "ingress" {
    for_each = toset(var.ssh_allowed_cidrs)

    content {
      description = "Restricted SSH access"
      from_port   = 22
      to_port     = 22
      protocol    = "tcp"
      cidr_blocks = [ingress.value]
    }
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = local.common_tags
}
```

Explanation

ALB security group

Allows incoming HTTP traffic from users.

EC2 security group

Allows incoming HTTP traffic only from the ALB security group.

It also allows SSH only from the IP addresses defined in our Terraform variables.

Why are we not allowing port 80 from 0.0.0.0/0 on EC2?

Because the assignment specifically says the service must not be directly accessible from the internet.

The Load Balancer should be the only entry point for application traffic.

PART 10 – Create the ECR Repository and Push the Image

10.1 What is Amazon ECR?

Amazon Elastic Container Registry is a managed Docker image registry.

It stores Docker images.

Think of it as a private Docker Hub hosted inside AWS.

Our process:

```
None
Local Docker Image
       |
       | Terraform pushes image
       v
Amazon ECR Repository
       |
       | EC2 pulls image
       v
Docker Container
```

10.2 Create ecr.tf

Location: `terraform/ecr.tf`

```
None
resource "aws_ecr_repository" "app" {
  name                 = var.project_name
  image_tag_mutability = "MUTABLE"

  force_delete = true

  image_scanning_configuration {
```

```
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }

  tags = local.common_tags
}

resource "docker_image" "app" {
  name =
"${aws_ecr_repository.app.repository_url}:${var.image_tag}"

  build {
    context = "${path.module}/../app"
  }

  keep_locally = true
}

resource "docker_registry_image" "app" {
  name = docker_image.app.name

  triggers = {
    image_id = docker_image.app.image_id
  }

  keep_remotely = false
}
```

Understand these resources separately

1. aws_ecr_repository

Creates the ECR repository in AWS.

2. docker_image

Builds the Docker image using the local application folder.

3. docker_registry_image

Pushes the built image into the ECR repository.

Why are we using Terraform for pushing?

The assignment specifically asks us to use Terraform to push the locally built image after creating the registry.

The Docker provider handles this operation.

Important: The local Docker image must already exist, and Docker Desktop must be running. Terraform also needs AWS ECR permissions to obtain an authentication token and upload the image.

PART 11 – Create IAM Role for EC2

11.1 Why do we need IAM?

Our EC2 instance needs permission to download the Docker image from ECR.

We should not place AWS access keys inside the EC2 instance.

Instead, we will attach an IAM role to EC2.

This is a more secure approach.

11.2 Create iam.tf

Location: `terraform/iam.tf`

```
None
resource "aws_iam_role" "ec2_role" {
  name = "${var.project_name}-ec2-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
```

```
    Statement = [{
      Effect = "Allow"

      Principal = {
        Service = "ec2.amazonaws.com"
      }

      Action = "sts:AssumeRole"
    }]
  })

  tags = local.common_tags
}

resource "aws_iam_role_policy_attachment" "ec2_ecr" {
  role = aws_iam_role.ec2_role.name

  policy_arn =
"arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

resource "aws_iam_instance_profile" "ec2_profile" {
  name = "${var.project_name}-instance-profile"

  role = aws_iam_role.ec2_role.name
}
```

Explanation

IAM Role

Creates an identity that EC2 can assume.

IAM Policy Attachment

Attaches the AWS-managed ECR read-only policy.

Instance Profile

Allows us to attach the IAM role to our EC2 instance.

What permission does EC2 receive?

EC2 can authenticate with ECR and pull container images.

It does not receive permission to push images.

That is intentional: only our local Terraform deployment needs to push the image.

PART 12 – Create the EC2 Instance and Deploy Docker

12.1 What are we creating?

We will create a single EC2 instance.

Configuration:

Setting	Value
AMI	Amazon Linux 2023
Instance type	t3.micro
Key Pair	microservice-key
VPC	Our custom VPC
Subnet	First public subnet

Security Group	EC2 security group
IAM Role	ECR read-only role
Docker	Installed automatically
Container	vk-microservice:v1
Application port	80

Check your AWS account's current pricing and free-tier eligibility before choosing the instance type.

12.2 Create the startup script

Location: `scripts/user_data.sh.tftpl`

Paste:

```
None
#!/bin/bash

set -euxo pipefail

REGION="${region}"
IMAGE_URI="${repository_url}:${image_tag}"
REGISTRY="${registry_url}"

# Install Docker and AWS CLI
dnf install -y docker awscli

# Start Docker
systemctl enable docker
```

```
systemctl start docker

# Authenticate with ECR
aws ecr get-login-password --region "$REGION" | \
docker login --username AWS --password-stdin "$REGISTRY"

# Pull Docker image from ECR
docker pull "$IMAGE_URI"

# Remove old container if present
docker rm -f vk-microservice || true

# Start application container
docker run -d \
  --name vk-microservice \
  --restart unless-stopped \
  -p 80:80 \
  "$IMAGE_URI"
```

What is this file?

This is an EC2 user-data startup script.

It automatically executes during the first boot of the EC2 instance.

What does it do?

1. Installs Docker.
2. Installs AWS CLI.
3. Starts Docker.
4. Authenticates with ECR.
5. Pulls our Docker image.
6. Starts the container.
7. Maps EC2 port 80 to container port 80.

Why are we using user data?

We do not need to manually log in to EC2 and install Docker.

Terraform creates the EC2 instance with the startup script already attached.

12.3 Create ec2.tf

Location: `terraform/ec2.tf`

Paste:

```
None
data "aws_ssm_parameter" "amazon_linux" {
  name =
"/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86
_64"
}

resource "aws_key_pair" "app_key" {
  key_name   = var.key_name
  public_key = file(var.public_key_path)

  tags = local.common_tags
}

resource "aws_instance" "app" {
  ami = data.aws_ssm_parameter.amazon_linux.value

  instance_type = var.instance_type

  subnet_id = aws_subnet.public[0].id

  vpc_security_group_ids = [
    aws_security_group.ec2_sg.id
  ]

  associate_public_ip_address = true

  key_name = aws_key_pair.app_key.key_name

  iam_instance_profile = aws_iam_instance_profile.ec2_profile.name

  user_data = templatefile(
```

```
    "${path.module}/../scripts/user_data.sh.tftpl",
    {
      region         = var.aws_region
      repository_url = aws_ecr_repository.app.repository_url
      registry_url   = split("/",
aws_ecr_repository.app.repository_url)[0]
      image_tag      = var.image_tag
    }
  )

  root_block_device {
    volume_size           = 8
    volume_type           = "gp3"
    encrypted             = true
    delete_on_termination = true
  }

  depends_on = [
    docker_registry_image.app,
    aws_iam_role_policy_attachment.ec2_ecr
  ]

  tags = merge(local.common_tags, {
    Name = "${var.project_name}-ec2"
  })
}
```

Understand the EC2 configuration

AMI

Uses the latest Amazon Linux 2023 AMI published through AWS Systems Manager Parameter Store.

Instance type

Defines the CPU and memory capacity.

Subnet

Places the EC2 instance in our first public subnet.

Security Group

Allows HTTP only from the ALB and SSH from approved IP addresses.

Key Pair

Allows SSH authentication.

IAM Instance Profile

Allows EC2 to pull images from ECR.

User Data

Installs Docker and runs our application container.

Depends_on

Makes sure the image is pushed to ECR and the IAM permission is attached before EC2 is created.

PART 13 – Create the Application Load Balancer

13.1 What is an ALB?

ALB stands for Application Load Balancer.

It receives application requests and forwards them to registered targets.

In our project:

```
None
Browser
   |
   | HTTP Request
   v
Application Load Balancer
   |
```

```
   | Forward traffic
   v
EC2 Instance
   |
   v
Docker Container
   |
   v
Nginx
   |
   v
index.html
```

Why do we need ALB?

The assignment says users should access the application only through a Load Balancer.

Therefore, users should access the ALB DNS name instead of the EC2 public IP.

13.2 Create alb.tf

Location: `terraform/alb.tf`

Paste:

None

```
resource "aws_lb" "app" {
  name               = "${var.project_name}-alb"
  internal           = false
  load_balancer_type = "application"

  security_groups = [
    aws_security_group.alb_sg.id
  ]

  subnets = aws_subnet.public[*].id
```

```
  tags = local.common_tags
}

resource "aws_lb_target_group" "app" {
  name     = "${var.project_name}-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id

  target_type = "instance"

  health_check {
    enabled             = true
    path                = "/"
    protocol            = "HTTP"
    matcher             = "200"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 2
    unhealthy_threshold = 3
  }

  tags = local.common_tags
}

resource "aws_lb_target_group_attachment" "app" {
  target_group_arn = aws_lb_target_group.app.arn

  target_id = aws_instance.app.id

  port = 80
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
```

```
  port     = 80
  protocol = "HTTP"

  default_action {
    type = "forward"

    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

Explanation

1. aws_lb

Creates the Application Load Balancer.

2. aws_lb_target_group

Defines which EC2 instances should receive application requests.

3. aws_lb_target_group_attachment

Registers our EC2 instance with the target group.

4. aws_lb_listener

Listens for incoming HTTP traffic on port 80 and forwards it to the target group.

5. Health Check

Checks whether our application is responding successfully.

If the application returns HTTP 200 at `/`, the target can become healthy.

PART 14 – Create Terraform Outputs

14.1 Create outputs.tf

Location: `terraform/outputs.tf`

Paste:

```
None
output "vpc_id" {
  value = aws_vpc.main.id
}

output "ecr_repository_url" {
  value = aws_ecr_repository.app.repository_url
}

output "ec2_instance_id" {
  value = aws_instance.app.id
}

output "ec2_public_ip" {
  value = aws_instance.app.public_ip
}

output "alb_dns_name" {
  value = aws_lb.app.dns_name
}

output "application_url" {
  value = "http://${aws_lb.app.dns_name}"
}
```

Why do we need outputs?

Terraform outputs display important resource information after deployment.

For example:

- VPC ID.
- ECR repository URL.
- EC2 instance ID.
- EC2 public IP.
- ALB DNS name.
- Application URL.

We can use the application URL to test our deployment.

PART 15 – Create terraform.tfvars

This file contains the actual values for our variables.

Location: `terraform/terraform.tfvars`

Paste:

```
None
aws_region  = "us-east-1"
project_name = "vk-microservice"

vpc_cidr = "10.0.0.0/16"

public_subnet_cidrs = [
  "10.0.1.0/24",
  "10.0.2.0/24"
]

instance_type = "t3.micro"

key_name = "microservice-key"

public_key_path =
"C:/Users/YOUR_WINDOWS_USERNAME/.ssh/microservice-key.pub"

ssh_allowed_cidrs = [
  "YOUR_PUBLIC_IP/32"
]

image_tag = "v1"
```

Important: Change two values

1. Windows username

Replace:

```
None
YOUR_WINDOWS_USERNAME
```

With your actual Windows username.

2. Your public IP

Open a browser and visit:

`https://checkip.amazonaws.com/`

Suppose it displays:

```
None
49.36.100.25
```

Then your Terraform value should be:

```
None
ssh_allowed_cidrs = [
  "49.36.100.25/32"
]
```

Do not use `0.0.0.0/0` for SSH.

The `/32` means only that individual IPv4 address is allowed.

If your public IP changes, update this value and apply Terraform again.

PART 16 – Initialize and Deploy Terraform

We have completed all Terraform files.

Now we will execute Terraform commands in the correct order.

16.1 Open the Terraform directory

Open PowerShell:

```
None
cd C:\terraform-microservice-project\terraform
```

Make sure Docker Desktop is running.

16.2 Initialize Terraform

Run:

```
None
terraform init
```

What happens?

Terraform downloads the required providers:

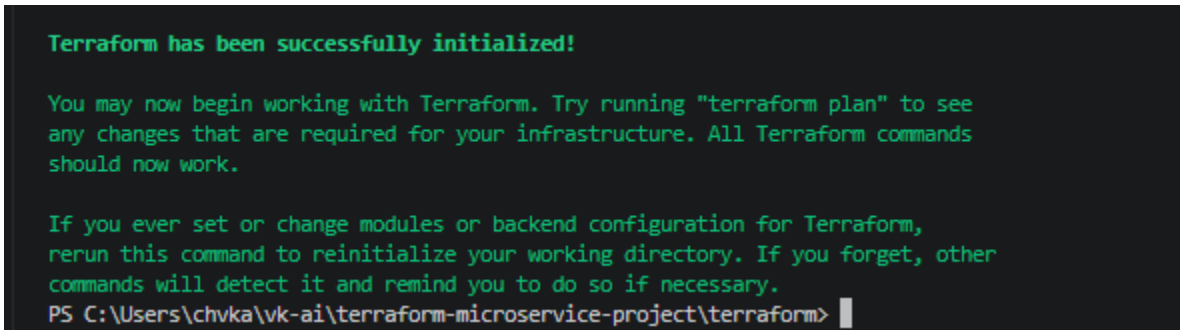
- AWS provider.
- Docker provider.

It also initializes the working directory.

Expected output:

```
None
Terraform has been successfully initialized!
```

If you get an error, do not continue until it is resolved.



16.3 Format Terraform files

Run:

```
None
terraform fmt -recursive
```

Why?

It formats Terraform files into a consistent structure.

```
PS C:\Users\chvka\vk-ai\terraform-microservice-project\terraform> terraform fmt -recursive
terraform.tfvars
PS C:\Users\chvka\vk-ai\terraform-microservice-project\terraform>
```

16.4 Validate Terraform files

Run:

```
None
terraform validate
```

Expected output:

```
None
Success! The configuration is valid.
```

```
PS C:\Users\chvka\vk-ai\terraform-microservice-project\terraform> terraform validate
Success! The configuration is valid.

PS C:\Users\chvka\vk-ai\terraform-microservice-project\terraform>
```

Why?

Terraform checks whether the configuration syntax and structure are valid.

Note that `validate` does not check whether AWS will accept every resource configuration or whether your credentials have sufficient permissions.

16.5 Generate Terraform execution plan

Run:

```
None
terraform plan
```

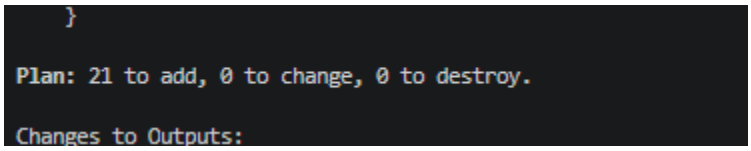
What happens?

Terraform checks your configuration and compares the desired infrastructure with the current infrastructure.

It shows which resources will be created.

You should see resources such as:

```
None
aws_vpc.main
aws_internet_gateway.igw
aws_subnet.public
aws_route_table.public
aws_security_group.alb_sg
aws_security_group.ec2_sg
aws_ecr_repository.app
aws_iam_role.ec2_role
aws_instance.app
aws_lb.app
aws_lb_target_group.app
aws_lb_listener.http
```



You should also see Docker image resources.

The exact number of resources can vary because some resources are created multiple times using `count`.

Important: Check that the SSH CIDR is your actual public IP and that the EC2 and ALB security group rules are correct.

16.6 Apply Terraform

Run:

```
None
terraform apply
```

Terraform will show the plan again.

Enter:

```
None
yes
```

Terraform now starts creating the infrastructure.

What will happen internally?

1. Create ECR repository
   AWS creates the Docker image repository.
2. Build and push Docker image
   Terraform's Docker provider builds the image from your local `app` folder and uploads it to ECR.
3. Create networking
   VPC, Internet Gateway, subnets and route tables are created.
4. Create security and IAM
   Security groups, IAM role and instance profile are created.
5. Create EC2
   Terraform creates the EC2 instance with the SSH key and IAM instance profile.
6. Deploy the Docker container
   EC2 executes the startup script, authenticates with ECR, pulls the image and starts the container.
7. Create Load Balancer
   ALB, target group and listener are created, and EC2 is registered.

Expected result

```
None
Apply complete!

Outputs:
```

```
alb_dns_name =
"vk-microservice-alb-....us-east-1.elb.amazonaws.com"

application_url =
"http://vk-microservice-alb-....us-east-1.elb.amazonaws.com"

ecr_repository_url =
"123456789012.dkr.ecr.us-east-1.amazonaws.com/vk-microservice"
```



Your actual DNS name and account ID will be different.

Note: Wait for the EC2 startup script to finish and for the ALB target health check to pass. Terraform finishing does not necessarily mean the application is already healthy.

PART 17 – Verify Resources in AWS Console

Now we will verify that Terraform created everything correctly.

17.1 Verify ECR

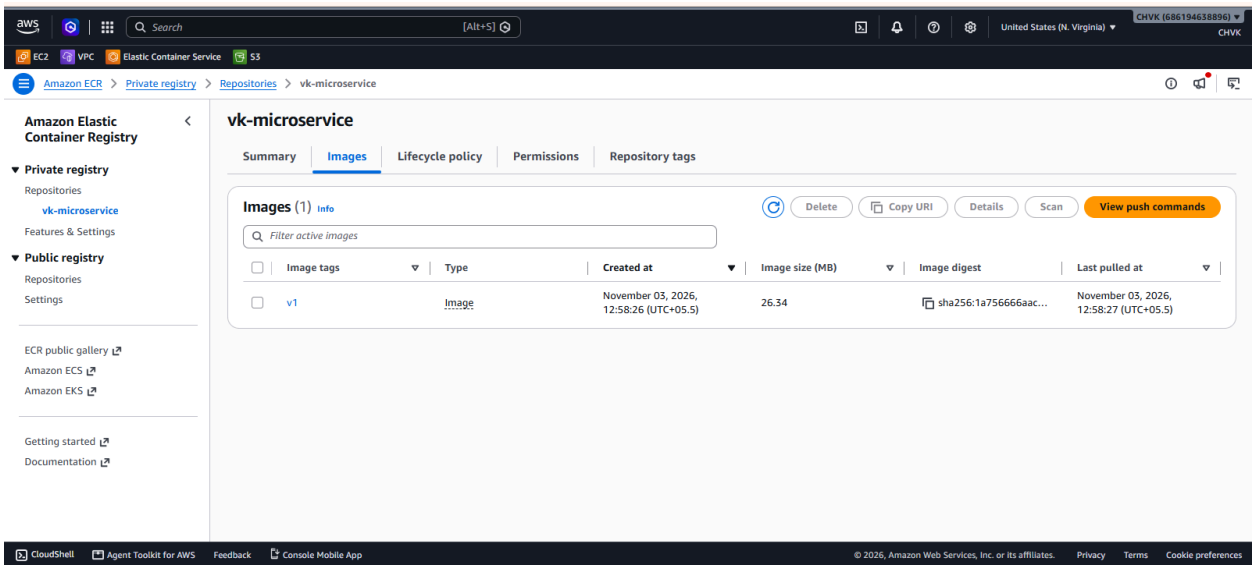
1. Open AWS Console.
2. Search for `ECR`.
3. Open Elastic Container Registry.
4. Select Private Registry.
5. Open Repositories.
6. Click `vk-microservice`.

You should see:

```
None
Repository: vk-microservice

Image:
    Tag: v1
```

This confirms that Terraform pushed the Docker image to ECR.



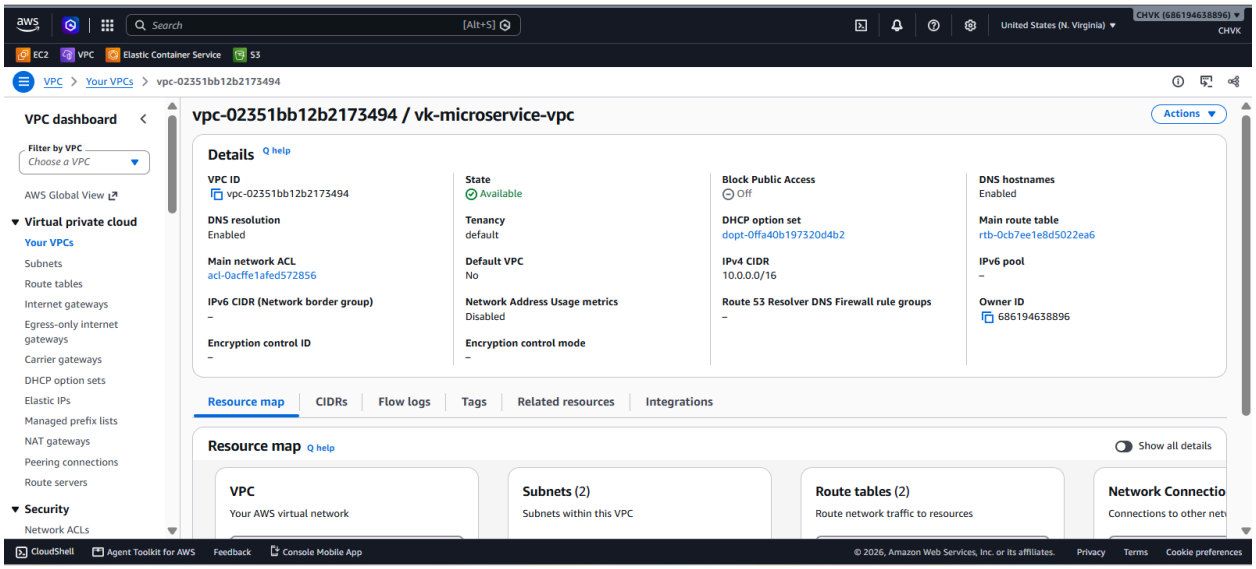
[How to]

17.2 Verify VPC

1. Open AWS Console.
2. Search for VPC.
3. Open Your VPCs.
4. Find `vk-microservice-vpc`.

Verify:

```
None
VPC CIDR: 10.0.0.0/16
State: Available
```

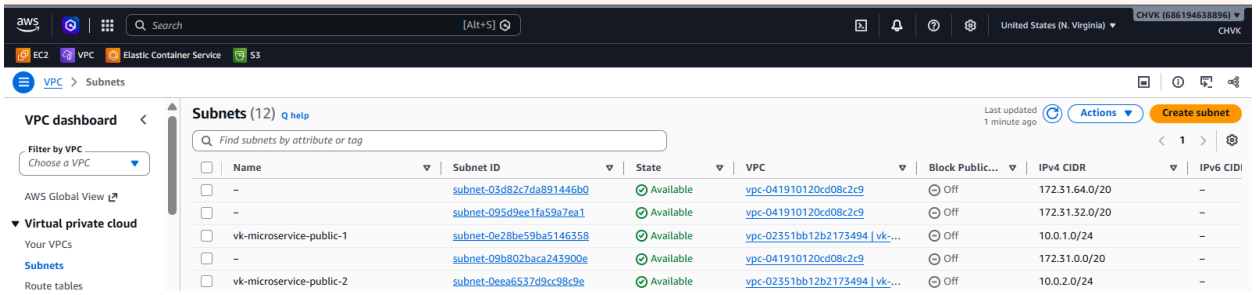


Next, open Subnets.

You should find two public subnets:

None

```
vk-microservice-public-1
vk-microservice-public-2
```



17.3 Verify Security Groups

Open:

EC2 → Network & Security → Security Groups.

Check:

ALB Security Group

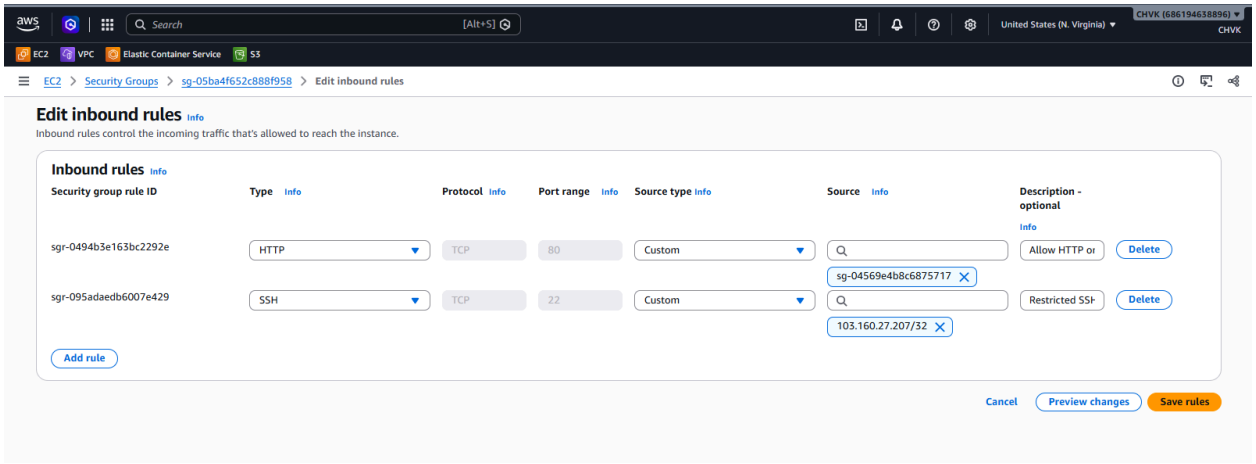
```
None
Inbound:
HTTP 80
Source: 0.0.0.0/0
```

EC2 Security Group

```
None
Inbound:
HTTP 80
Source: ALB Security Group

SSH 22
Source: Your Public IP/32
```

If EC2 port 80 is open to `0.0.0.0/0`, the configuration is incorrect for this assignment.



17.4 Verify IAM Role

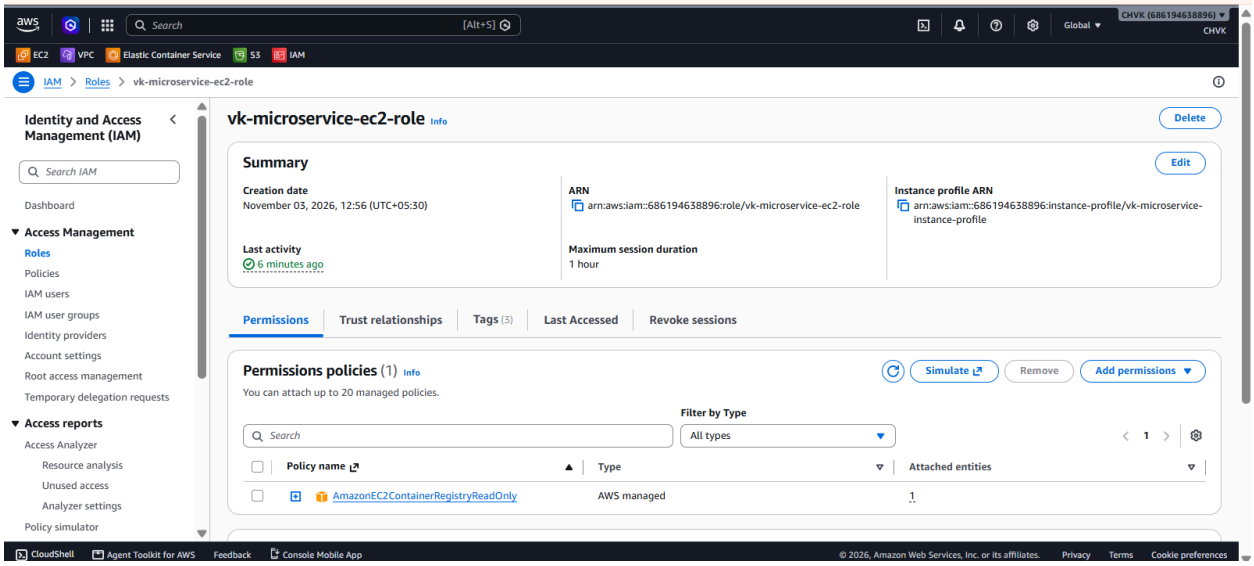
1. Open AWS Console.
2. Search for IAM.
3. Open Roles.
4. Search `vk-microservice-ec2-role`.
5. Open the role.

Check that the policy is attached:

None

```
AmazonEC2ContainerRegistryReadOnly
```

This confirms that EC2 has permission to pull the Docker image.



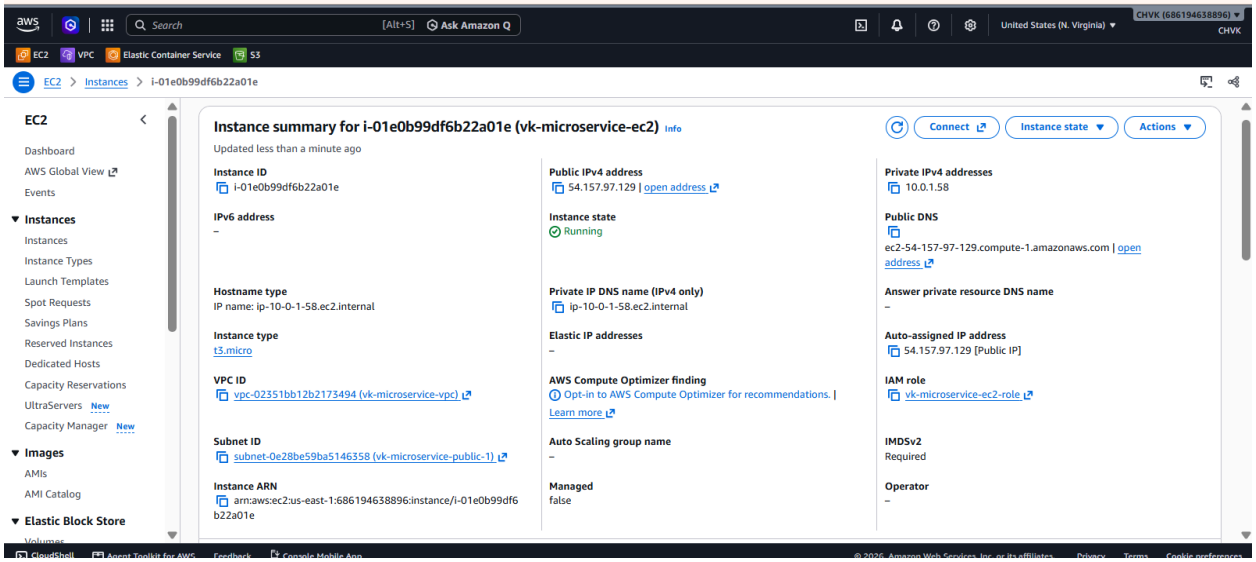
17.5 Verify EC2

1. Open AWS Console.
2. Search for EC2.
3. Click Instances.
4. Find `vk-microservice-ec2`.

Verify:

Setting	Expected
Instance state	Running
Instance type	t3.micro

AMI	Amazon Linux 2023
Key pair	microservice-key
VPC	Our custom VPC
Security Group	vk-microservice-ec2-sg



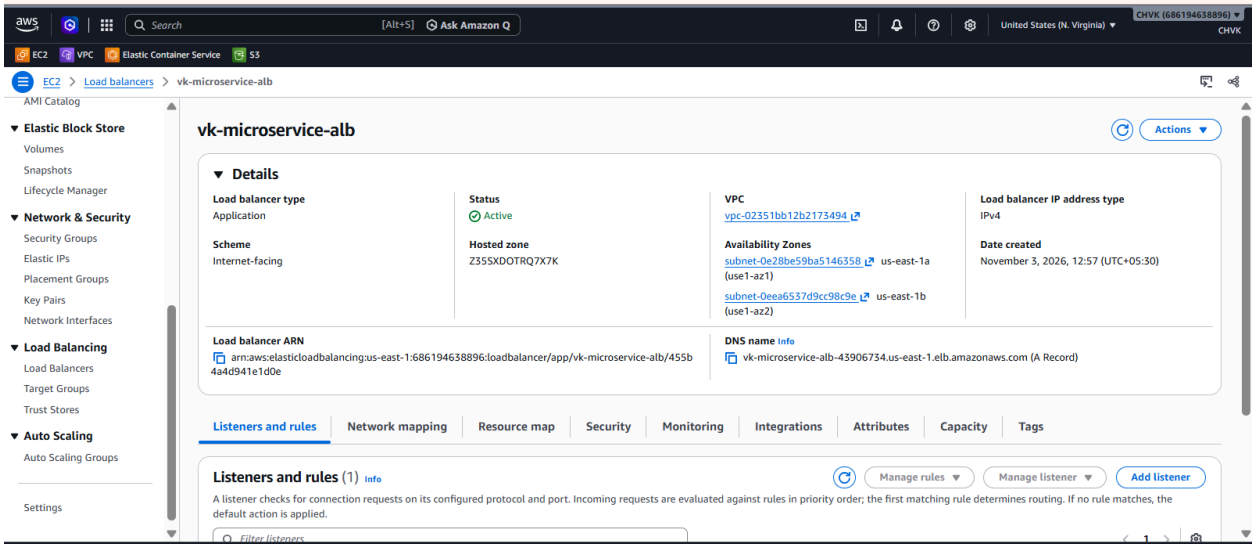
17.6 Verify Load Balancer

1. Open AWS Console.
2. Search for EC2.
3. Under Load Balancing, select Load Balancers.
4. Find `vk-microservice-alb`.

Check:

```
None
Type: Application
Scheme: Internet-facing
```

```
State: Active
```



Next, open Target Groups.

Find:

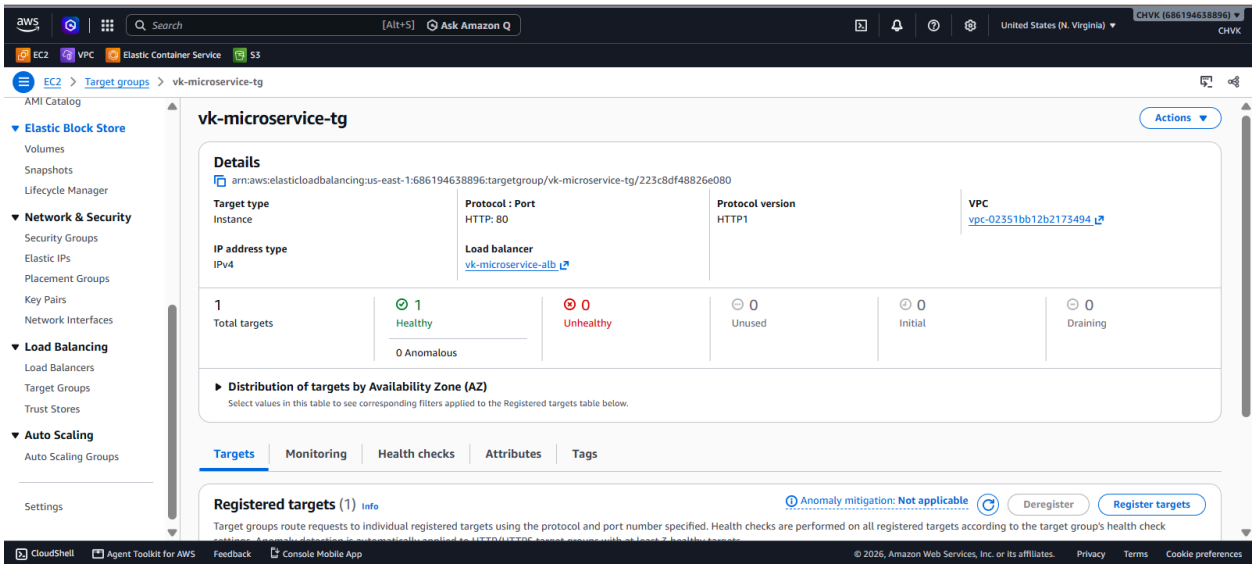
```
None
vk-microservice-tg
```

Open the Targets tab.

Expected:

Healthy

```
None
EC2 Instance: Registered
Port: 80
Health: Healthy
```



If it shows `Unhealthy`, see the troubleshooting section below.

PART 18 – Test the Application

This is the most important final test.

18.1 Get the ALB DNS name

In PowerShell:

```
None
terraform output alb_dns_name
```

Or:

```
None
terraform output application_url
```

Example:

```
None
http://vk-microservice-alb-123456789.us-east-1.elb.amazonaws.com
```

```
PS C:\Users\chvka\vk-ai\terraform-microservice-project\terraform> terraform output alb_dns_name
"vk-microservice-alb-43906734.us-east-1.elb.amazonaws.com"
PS C:\Users\chvka\vk-ai\terraform-microservice-project\terraform> terraform output application_url
"http://vk-microservice-alb-43906734.us-east-1.elb.amazonaws.com"
```

18.2 Open the application

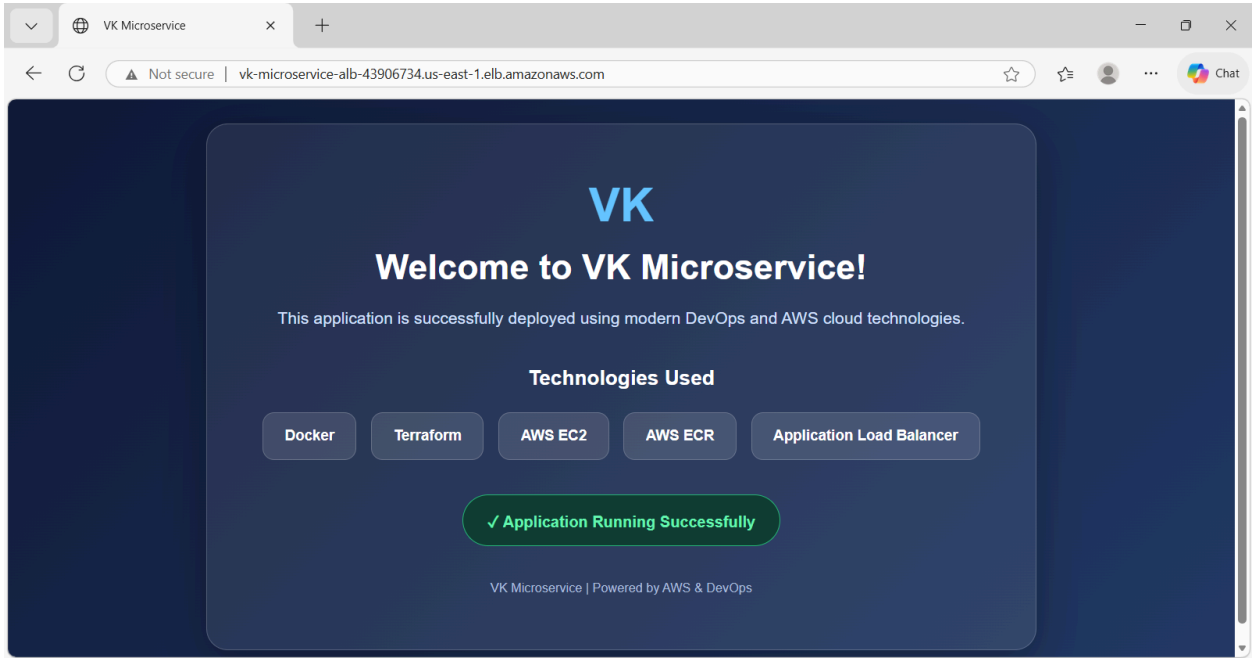
Copy your actual application URL.

Open it in Chrome.

Expected output:

VK Microservice

HTTP 200



18.3 Test using PowerShell

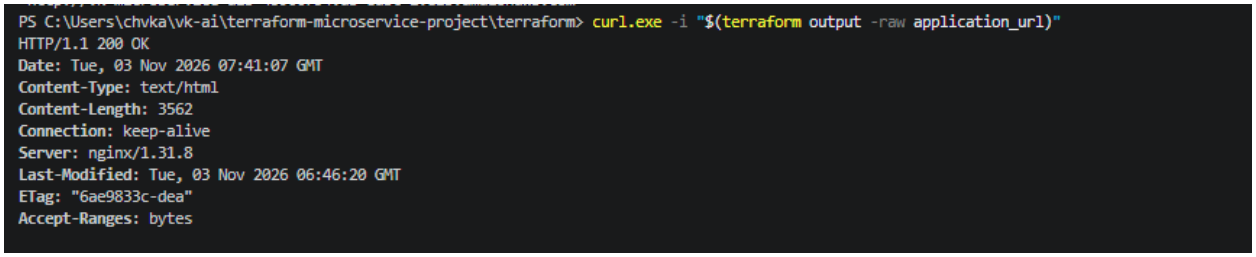
Run:

```
None
curl.exe -i "$(terraform output -raw application_url)"
```

Expected:

```
None
HTTP/1.1 200 OK
Content-Type: text/html
```

You should also see the HTML content.



18.4 Test direct EC2 access

Now copy the EC2 public IP:

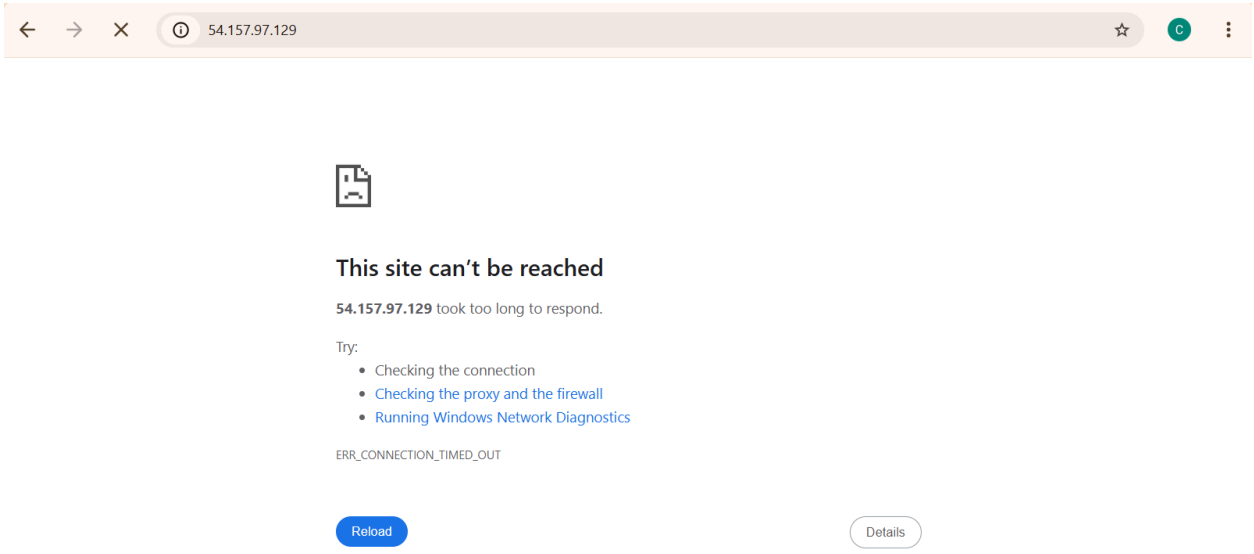
```
None
terraform output ec2_public_ip
```

Try opening:

```
None
http://EC2_PUBLIC_IP
```

Expected:

```
None
Connection timed out
```



This is intentional.

The EC2 security group only allows port 80 from the ALB security group.

Important distinction: The EC2 instance has a public IP for this simplified lab, but its application port is not publicly accessible. Its SSH port is restricted to your specified IP.

PART 19 – Connect to EC2 Using SSH

We also need to verify that our SSH key works.

Open PowerShell:

```
None
cd C:\terraform-microservice-project
```

Run:

```
None
ssh -i "$env:USERPROFILE\.ssh\microservice-key"
ec2-user@EC2_PUBLIC_IP
```

Replace `EC2_PUBLIC_IP` with the actual IP from Terraform.

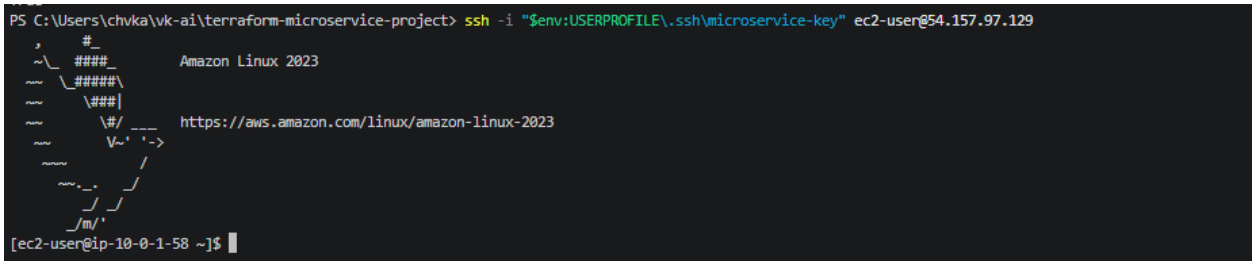
If asked:

```
None
Are you sure you want to continue connecting?
```

Enter:

```
None
yes
```

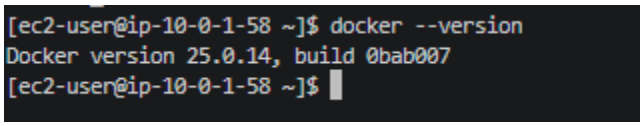
You should connect to the Amazon Linux server.



19.1 Check Docker installation

Inside EC2:

```
None
docker --version
```



19.2 Check running containers

```
None
sudo docker ps
```

Expected:

None

```
CONTAINER ID   IMAGE
xxxxxxxx       ECR_IMAGE_URI
```

```
[ec2-user@ip-10-0-1-58 ~]$ sudo docker ps
CONTAINER ID   IMAGE                                                      COMMAND                  CREATED          STATUS          PORTS                                     NAMES
9c54708e7944   686194638896.dkr.ecr.us-east-1.amazonaws.com/vk-microservice:v1   "/docker-entrypoint.…"   16 minutes ago   Up 16 minutes   0.0.0.0:80->80/tcp, :::80->80/tcp   vk-microservice
[ec2-user@ip-10-0-1-58 ~]$
```

19.3 Check the application logs

None

```
sudo docker logs vk-microservice
```

```
[ec2-user@ip-10-0-1-58 ~]$ sudo docker logs vk-microservice
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/11/03 07:29:48 [notice] 1#1: using the "epoll" event method
2026/11/03 07:29:48 [notice] 1#1: nginx/1.31.8
2026/11/03 07:29:48 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/11/03 07:29:48 [notice] 1#1: OS: Linux 6.18.56-125.164.amzn2023.x86_64
2026/11/03 07:29:48 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 32768:65536
2026/11/03 07:29:48 [notice] 1#1: start worker processes
2026/11/03 07:29:48 [notice] 1#1: start worker process 30
2026/11/03 07:29:48 [notice] 1#1: start worker process 31
10.0.1.138 - - [03/Nov/2026:07:29:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:30:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:30:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:30:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:31:07 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:31:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:31:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:31:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:32:07 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:32:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:32:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:32:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:33:07 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:33:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:33:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:33:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:34:07 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:34:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:34:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:34:38 +0000] "GET / HTTP/1.1" 200 3562 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:120.0) Gecko/20100101 Firefox/120.0" "209.99.187.148"
10.0.1.138 - - [03/Nov/2026:07:34:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:35:07 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:35:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:35:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:35:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:36:07 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:36:24 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.2.6 - - [03/Nov/2026:07:36:37 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
10.0.1.138 - - [03/Nov/2026:07:36:54 +0000] "GET / HTTP/1.1" 200 3562 "-" "ELB-HealthChecker/2.0" "-"
```

19.4 Verify the application locally on EC2

None

```
curl http://localhost
```

You should receive the HTML page.

```
[ec2-user@ip-10-0-1-58 ~]$ curl http://localhost

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>VK Microservice</title>

    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            background: linear-gradient(135deg, #101a38, #243b67);
            color: white;
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            padding: 25px;
        }

        .container {
            max-width: 850px;
            width: 100%;
            background: rgba(255, 255, 255, 0.08);
            border: 1px solid rgba(255, 255, 255, 0.2);
            border-radius: 22px;
            padding: 55px 35px;
            text-align: center;
            box-shadow: 0 15px 45px rgba(0, 0, 0, 0.3);
        }

        .logo {
            font-size: 48px;
            font-weight: bold;
            color: #65c7ff;
            margin-bottom: 15px;
        }

        h1 {
            font-size: 36px;
            margin-bottom: 18px;
```

19.5 Check user-data logs if deployment fails

None

```
sudo cat /var/log/cloud-init-output.log
```

This is especially useful when Docker installation or the ECR image pull fails.

```
[ec2-user@ip-10-0-1-58 ~]$ sudo cat /var/log/cloud-init-output.log
Cloud-init v. 22.2.2 running 'init' at Tue, 03 Nov 2026 07:28:38 +0000. Up 5.71 seconds.
ci-info: ++++++++++++++++++++++++++++++++++++Net device info++++++++++++++++++++++++++++++++++++
ci-info: +--------+------+---------------------------+---------------+--------+-------------------+
ci-info: | Device |  Up  |          Address          |      Mask     | Scope  |     Hw-Address    |
ci-info: +--------+------+---------------------------+---------------+--------+-------------------+
ci-info: |  ens5  | True |         10.0.1.58         | 255.255.255.0 | global | 02:ff:c5:21:17:bf |
ci-info: |  ens5  | True | fe80::ff:c5ff:fe21:17bf/64 |       .       |  link  | 02:ff:c5:21:17:bf |
ci-info: |   lo   | True |         127.0.0.1         |   255.0.0.0   |  host  |         .         |
ci-info: |   lo   | True |          ::1/128          |       .       |  host  |         .         |
ci-info: +--------+------+---------------------------+---------------+--------+-------------------+
ci-info: ++++++++++++++++++++++++++++Route IPv4 info++++++++++++++++++++++++++++
ci-info: +-------+-------------+----------+-----------------+-----------+-------+
ci-info: | Route | Destination | Gateway  |     Genmask     | Interface | Flags |
ci-info: +-------+-------------+----------+-----------------+-----------+-------+
ci-info: |   0   |   0.0.0.0   | 10.0.1.1 |     0.0.0.0     |    ens5   |   UG  |
ci-info: |   1   |   10.0.0.2  | 10.0.1.1 | 255.255.255.255 |    ens5   |  UGH  |
ci-info: |   2   |   10.0.1.0  | 0.0.0.0  |  255.255.255.0  |    ens5   |   U   |
ci-info: |   3   |   10.0.1.1  | 0.0.0.0  | 255.255.255.255 |    ens5   |   UH  |
ci-info: +-------+-------------+----------+-----------------+-----------+-------+
ci-info: ++++++++++++++++++Route IPv6 info++++++++++++++++++
ci-info: +-------+-------------+---------+-----------+-------+
ci-info: | Route | Destination | Gateway | Interface | Flags |
ci-info: +-------+-------------+---------+-----------+-------+
ci-info: |   0   |  fe80::/64  |    ::   |    ens5   |   U   |
ci-info: |   2   |    local    |    ::   |    ens5   |   U   |
ci-info: |   3   |  multicast  |    ::   |    ens5   |   U   |
ci-info: +-------+-------------+---------+-----------+-------+
Generating public/private ed25519 key pair.
Your identification has been saved in /etc/ssh/ssh_host_ed25519_key
Your public key has been saved in /etc/ssh/ssh_host_ed25519_key.pub
The key fingerprint is:
SHA256:rmMhNjuZNC47WySVmwxgvYzqxmR30WNtY1NdwytVKDs root@ip-10-0-1-58.ec2.internal
The key's randomart image is:
+--[ED25519 256]--+
|  o.        . o+o|
| . ...     ....o.|
|   o+.. . .  o. .|
|  ..o= = *  E. . |
| .. . * S o  ..  |
|.o +*...         |
|= .+oB ..        |
| +o.* o.         |
|. o= o..         |
+----[SHA256]-----+
```

To exit EC2:

```
None
exit
```

PART 20 – Troubleshooting Common Errors

During this project, you may encounter some errors. Here are the common problems and solutions.

20.1 Docker daemon error

Error:

```
None
Cannot connect to the Docker daemon
```

Reason: Docker Desktop is not running.

Solution:

1. Open Docker Desktop.
2. Wait for Docker Engine to start.
3. Run:

```
None
docker info
```

4. Run Terraform again.

20.2 ECR authentication error

Error:

```
None
no basic auth credentials
```

Reason: Terraform cannot authenticate with ECR.

Solution:

Verify your AWS credentials:

```
None
aws sts get-caller-identity
```

Verify ECR authorization:

```
None
aws ecr get-authorization-token --region us-east-1
```

Make sure your AWS identity has the required ECR permissions.

20.3 Docker image not found

Error:

```
None
repository does not exist
```

Reason: The image was not successfully pushed to ECR.

Solution:

Check:

```
None
aws ecr describe-images --repository-name vk-microservice --region
us-east-1
```

Make sure the image tag `v1` exists.

20.4 EC2 cannot pull the image

Possible reason:

- IAM role is missing.
- ECR read permission is missing.
- Docker installation failed.
- Internet connectivity is unavailable.
- Image was not pushed.

Check the EC2 startup log:

```
None
sudo cat /var/log/cloud-init-output.log
```

Check the IAM role attached to the instance.

20.5 ALB target is unhealthy

Possible reasons:

- Docker container is not running.
- Nginx is not running.
- EC2 security group does not allow port 80 from the ALB security group.
- Application is not responding on `/`.

Connect to EC2:

```
None
sudo docker ps -a
```

Check:

```
None
sudo docker logs vk-microservice
```

Test:

```
None
curl http://localhost
```

If localhost works but ALB health checks fail, inspect the EC2 security group and target group settings.

20.6 SSH connection timeout

Possible reasons:

- Your public IP changed.
- SSH port 22 is not allowed.
- Wrong private key.
- Wrong EC2 username.
- Incorrect security group.

Check your current IP and update:

```
None
ssh_allowed_cidrs = [
  "YOUR_CURRENT_IP/32"
]
```

Then:

```
None
terraform apply
```

PART 21 – Important Terraform Commands

Keep this command reference for your project.

Command	Purpose
`terraform init`	Initialize Terraform
`terraform fmt -recursive`	Format Terraform files
`terraform validate`	Validate configuration
`terraform plan`	Preview infrastructure changes
`terraform apply`	Create/update resources

`terraform show`	Display current state
`terraform state list`	List managed resources
`terraform output`	Display output values
`terraform refresh`	Not a standalone modern workflow; use plan/apply to refresh state
`terraform destroy`	Destroy managed infrastructure

If you modify your application

For example, if you update `index.html`:

1. Modify the HTML file.
2. Run `terraform apply` again.

The Docker provider will rebuild the image when its build context changes, and the registry image resource will push the updated image.

However, changing the image tag or image does not automatically guarantee that an already-running EC2 container will restart with the new version. For this lab, if you need to redeploy a changed image reliably, replace the EC2 instance through Terraform after updating the image, or implement a controlled deployment mechanism. This is one of the reasons production deployments normally use a CI/CD pipeline or an orchestration platform.

PART 22 – Destroy All AWS Resources

Important – Avoid unnecessary AWS charges

After completing your assignment and taking screenshots, destroy the resources if you no longer need them.

Open PowerShell:

```
None
cd C:\terraform-microservice-project\terraform
```

Run:

```
None
terraform destroy
```

Terraform displays the resources that will be deleted.

Enter:

```
None
yes
```

Terraform will destroy:

- Application Load Balancer.
- Target group and listener.
- EC2 instance.
- ECR repository and its images.
- IAM role and instance profile.
- Security groups.
- VPC.
- Internet Gateway.
- Subnets.
- Route table and associations.
- AWS key pair.

The local SSH key files and your local Docker image are not deleted.

After destruction, verify in AWS Console that the resources are gone.

Important: Do not manually delete the Terraform state file or ECR repository while Terraform is managing them. Use Terraform to destroy resources so that the state remains consistent.

PART 23 – How to Explain This Project in an Interview

You can use this explanation when an interviewer asks about your Terraform project.

Project Name: Containerized Microservice Deployment on AWS Using Terraform

Project Explanation

I worked on a containerized microservice deployment project using Terraform and AWS.

The objective was to deploy a simple Nginx web application inside a Docker container on an AWS EC2 instance.

First, I created a static HTML application and a Dockerfile. I built the Docker image locally and used Terraform with the Docker provider to push the image into Amazon Elastic Container Registry.

Next, I used Terraform to provision the AWS infrastructure, including a VPC, Internet Gateway, public subnets, route tables, security groups, IAM role, EC2 instance and Application Load Balancer.

I configured an IAM role with ECR read-only permissions so that the EC2 instance could securely authenticate with ECR and pull the Docker image without storing AWS credentials on the server.

I used EC2 user data to install Docker, pull the image and start the container automatically.

Finally, I configured an Application Load Balancer with a target group and health checks to route user requests to the EC2 instance.

For security, I configured the EC2 security group to allow application traffic only from the ALB security group and restricted SSH access to my specified public IP address.

I used Terraform variables, locals and separate configuration files to organize the infrastructure code.

This project helped me understand Infrastructure as Code, Docker image management, AWS networking, IAM, EC2 deployment, Load Balancing and application security.

Important interview questions

1. Why did you use ECR instead of Docker Hub?

ECR is AWS's managed container registry. It integrates with IAM and allows EC2 instances to authenticate and pull images using IAM roles.

2. Why did you use an IAM role for EC2?

To avoid storing permanent AWS access keys on the EC2 instance. The role provides temporary credentials and the required ECR permissions.

3. Why did you use an Application Load Balancer?

To provide a single entry point for users and forward HTTP requests to the application running on EC2.

4. How did you restrict direct EC2 access?

I configured the EC2 security group to allow port 80 only from the ALB security group. SSH port 22 is restricted to a specific public IP address.

5. Why did you use Terraform?

Terraform allows us to create, manage and reproduce infrastructure using configuration files rather than manually creating resources in AWS Console.

6. How does the Docker image reach EC2?

Terraform pushes the locally built image into ECR. EC2 uses its IAM role to authenticate with ECR, pull the image and run the container.

Final understanding – What have you actually built?

You have implemented a small but complete cloud deployment workflow.

Stage	Technology	What you learned
Application	HTML	Static web application
Packaging	Docker	Containerization

Registry	AWS ECR	Image storage
Infrastructure	Terraform	Infrastructure as Code
Networking	AWS VPC	Network isolation and routing
Compute	AWS EC2	Virtual machine deployment
Security	IAM + Security Groups	Access control
Traffic	AWS ALB	Application load balancing
Deployment	EC2 User Data	Automated container deployment

One production limitation to understand: This assignment uses a single EC2 instance. It demonstrates the required deployment architecture, but it does not provide application-level high availability. If that instance fails, the application becomes unavailable. A production design would normally use multiple instances across Availability Zones, preferably with Auto Scaling or ECS.